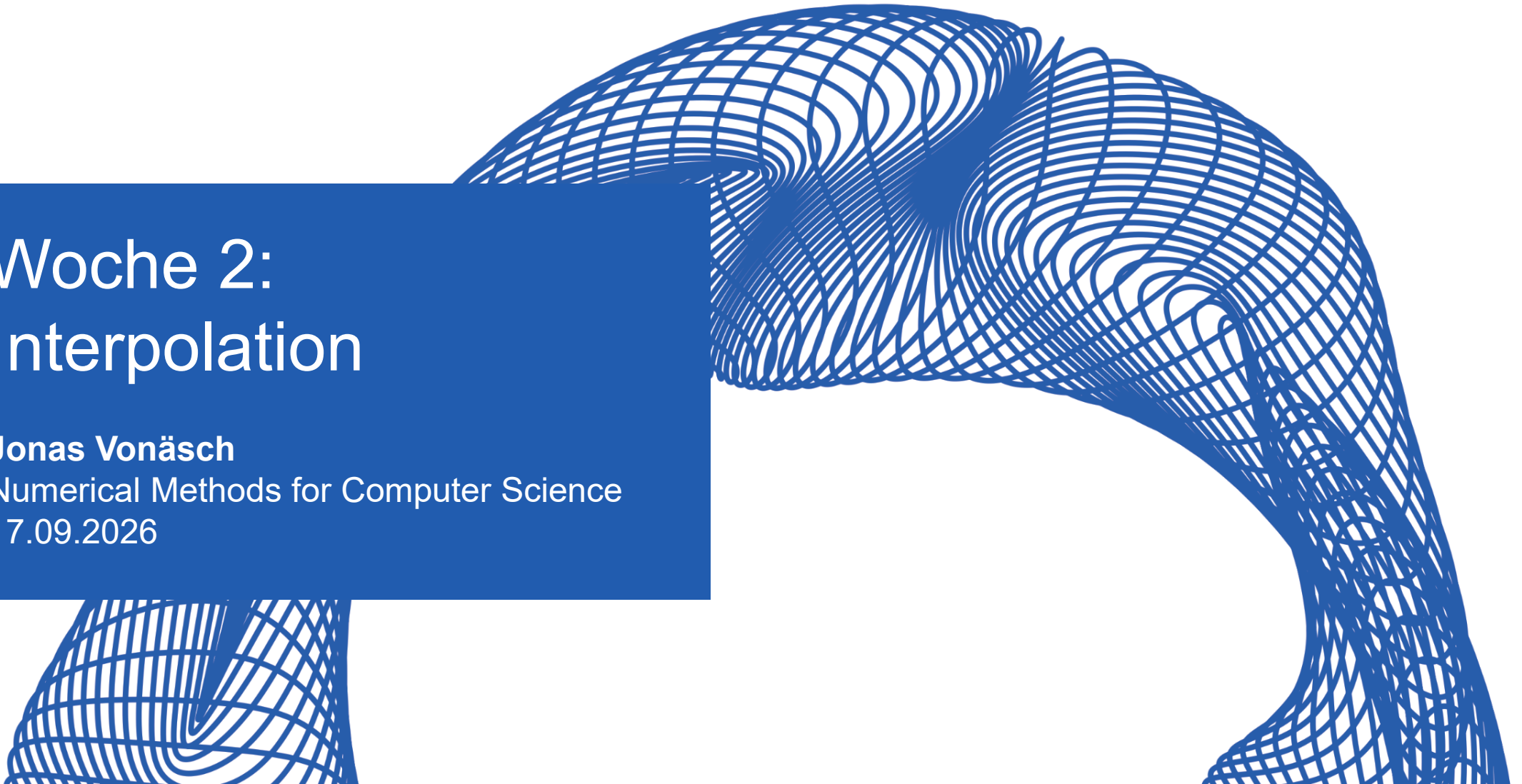


Woche 2: Interpolation

Jonas Vonäsch

Numerical Methods for Computer Science

17.09.2026



Inhaltsverzeichnis/Agenda

1. Theorie-Recap
 1. Newton
 2. Lagrange / Baryzentrische Formel
 3. Runge-Phänomen / Lebesgue
 4. Chebyshev-Interpolation
2. Nachbesprechung Aufgaben 1
3. Vorbesprechung Aufgaben 2

Interpolation

Interpolation

Ziel der Interpolation ist es, eine Funktion $\tilde{f}$ zu einer Menge gegebener Datenpunkte zu finden:

$$\begin{bmatrix} x_0 & x_1 & \cdots & x_n \\ y_0 & y_1 & \cdots & y_n \end{bmatrix}, x_i \in \mathbb{R}, y_i \in \mathbb{R}$$

Interpolationsbedingung:

$$\tilde{f}(x_i) = y_i \text{ for all } i$$

Jede Interpolation nimmt einen Unterraum $\mathcal{V}_n$ des Raumes aller Funktionen $\mathcal{V}$ und eine Basis b dieses Unterraumes mit $\text{span}\{b_1, \dots, b_n\} = \mathcal{V}_n$ und projiziert die Datenpunkte auf diese Basis:

$$f(x) \approx f_n(x) = \sum_{j=1}^n \alpha_j b_j(x)$$

Achtung: Interpolation ist *keine* Regression. $\tilde{f}$ muss durch alle Datenpunkte gehen.

Polynominterpolation

Polynominterpolation: Monombasis

Der Raum aller polynome sieht wie folgt aus:

$$\mathcal{P}_k := \{x \mapsto \alpha_k x^k + \alpha_{k-1} x^{k-1} + \dots + \alpha_1 x + \alpha_0, \alpha_j \in \mathbb{K}\}$$

Eine Basis dieses Raumes ist die sog. Monombasis:

$$\{x \mapsto 1, x \mapsto x, x \mapsto x^2, \dots, x \mapsto x^k\}$$

Man spricht bei $\alpha_k x^k + \alpha_{k-1} x^{k-1} + \dots + \alpha_1 x + \alpha_0$ von der *Monomdarstellung* eines Polynoms.

Polynominterpolation: Monombasis

Polynominterpolation in der Monombasis entspricht der Lösung des Gleichungssystems:

Vandermonde-matrix 

$$\begin{bmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Nutzt: `np.polyfit`

Monombasis ist nicht wirklich effizient!

Newton-Interpolation

Polynominterpolation: Newton

Idee: mit einer Stützstelle anfangen:

$$p_0(x_0) = y_0$$

Aufbauen:

$$p_1(x) = p_0(x) + (x - x_0) \frac{(y_1 - y_0)}{(x_1 - x_0)}$$

$$p_2(x) = p_1(x) + \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} (x - x_0)(x - x_1)$$

Daraus ergibt sich die Basis:

$$N_0(x) := 1, N_1(x) := x - x_0, N_2(x) := (x - x_0)(x - x_1), \dots, N_n(x) := \prod_{i=0}^{n-1} (x - x_i)$$

Polynominterpolation: Newton

Beobachte die Regelmässigkeit:

$$p_0(x_0) = y_0$$

$$p_1(x) = p_0(x) + (x - x_0) \frac{(y_1 - y_0)}{(x_1 - x_0)}$$

$$p_2(x) = p_1(x) + \frac{\frac{y_2 - y_1}{x_2 - x_1} - \frac{y_1 - y_0}{x_1 - x_0}}{x_2 - x_0} (x - x_0)(x - x_1)$$



Polynominterpolation: Newton

Die wiederholten Terme nennen wir **Dividierte Differenzen**:

$$y[x_i] = y_i \text{ und } y[x_i, \dots, x_{i+k}] = \frac{y[x_{i+1}, \dots, x_{i+k}] - y[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}.$$

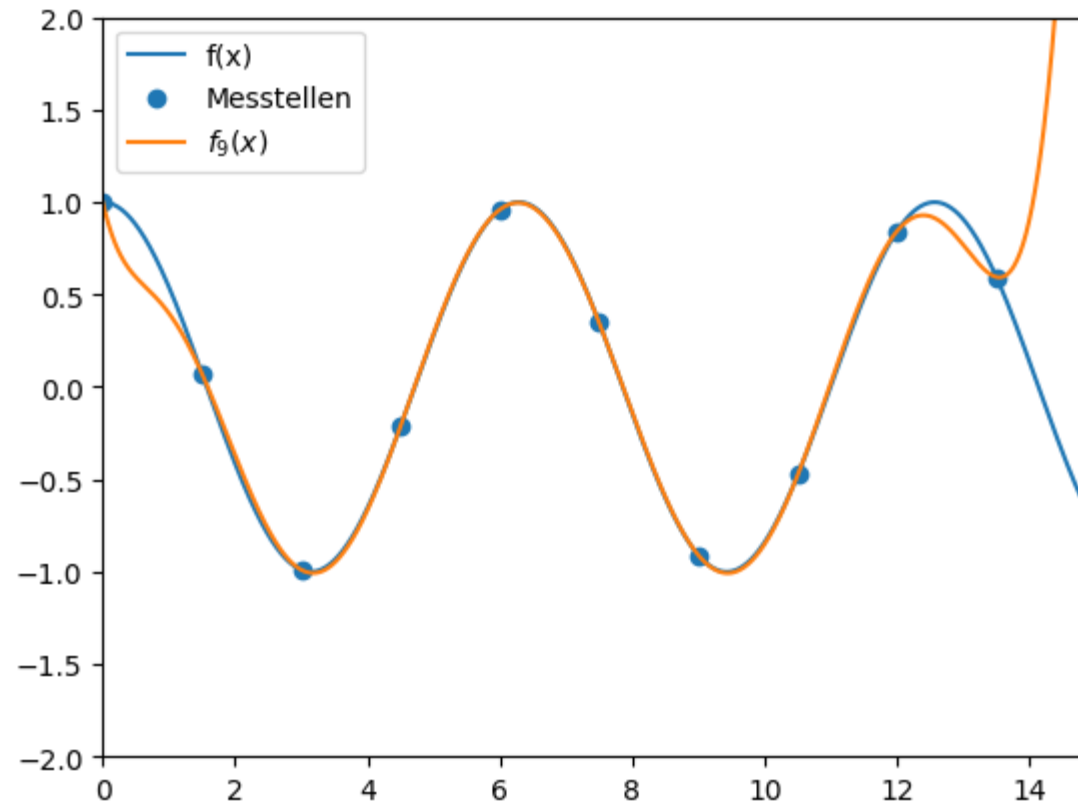
Erinnert an das Richardson-Schema:

$$\begin{array}{c|c} x_0 & y[x_0] \\ & > y[x_0, x_1] \\ x_1 & y[x_1] > y[x_0, x_1, x_2] \\ & > y[x_1, x_2] > y[x_0, x_1, x_2, x_3] \\ x_2 & y[x_2] > y[x_1, x_2, x_3] \\ & > y[x_2, x_3] \\ x_3 & y[x_3], \end{array}$$

Tipp: *sehr* ähnlich wie Richardson in Python.

$y[x_0], y[x_0, x_1], \dots, y[x_0, x_1, \dots, x_n]$ entsprechen nun genau den Koeffizienten β .

Polynominterpolation: Newton Visuell



Lagrange-Interpolation

Polynominterpolation: Lagrange

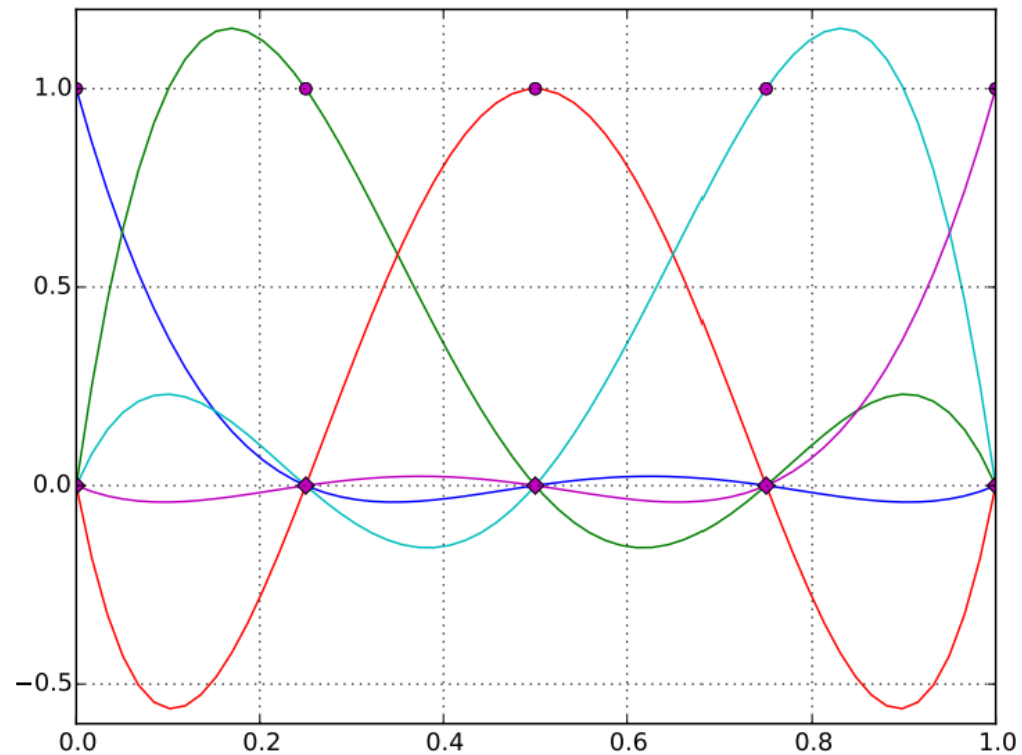
Idee: Was, wenn für Basisfunktion $\ell_n(x)$ jeweils $\ell_i(x_i) = 1$ und für $i \neq j$ gilt $\ell_i(x_j) = 0$?


$$p(x) = \sum_{i=0}^n y_i \ell_i(x) \quad \text{mit} \quad \ell_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

Polynominterpolation: Lagrange

Die Lagrange-Basis produziert quasi “Einheitsvektoren”:

1. $\ell_i(x_j) = 0$ für alle $j \neq i$,
2. $\ell_i(x_i) = 1$ für alle i ,
3. $\text{grad } \ell_i = n$ für alle i .
4. $\sum_{k=0}^n \ell_k(x) = 1$ und $\sum_{k=0}^n \ell_k^{(m)}(x) = 0$ für $m > 0$.



Polynominterpolation: Lagrange Baryzentrisch (Skript)

Wenn wir $L(x) = (x - x_0)(x - x_1) \cdots (x - x_n)$ und

$$\lambda_k = \prod_{j \neq k} \frac{1}{x_k - x_j} = \frac{1}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$$

definieren, dann lässt sich p umschreiben zu

$$p(x) = L(x) \sum_{k=0}^n \frac{\lambda_k}{x - x_k} y_k .$$

Für das konstante Polynom 1 ergibt das

$$1 = L(x) \sum_{k=0}^n \frac{\lambda_k}{x - x_k} ,$$

was zur baryzentrischen Formel für das Interpolationspolynom führt:

$$p(x) = \frac{\sum_{k=0}^n \frac{\lambda_k}{x - x_k} y_k}{\sum_{k=0}^n \frac{\lambda_k}{x - x_k}} .$$

Polynominterpolation: Fehler

Polynominterpolation: Runge-Phänomen

Fehler einer Polynominterpolation hängt von der Wahl der Stützstellen ab:

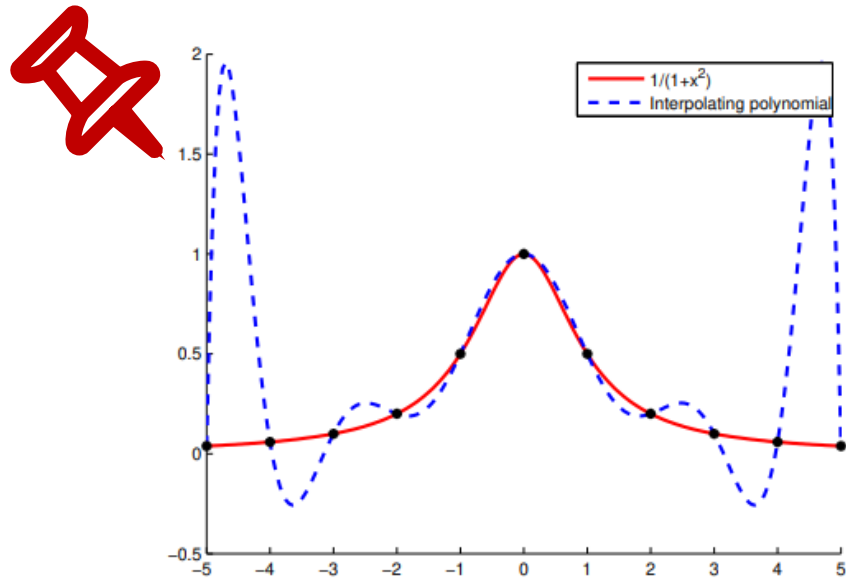


Figure 2.2: Interpolationspolynom

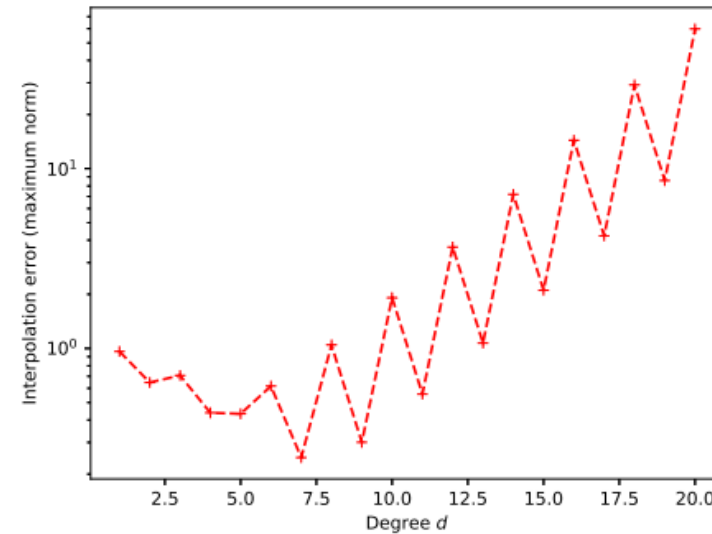


Figure 2.3: $\|f - p\|_\infty$ auf $[-5, 5]$

Lebesgue-Konstante

$$\Lambda_n := \max_{x_j \in [a,b]} \sum_{i=0}^n |\ell_i(x_j)|$$

Stark abhängig von der Wahl aller Stützstellen $x_1, \dots, x_n$.

$$\max_{x \in [a,b]} |p(x) - \tilde{p}(x)| \leq \Lambda_n \max_{i=0, \dots, n} |y_i - \tilde{y}_i|$$

Für beliebige polynome limitiert Λ_n den Fehler!

Äquidistant ist katastrophal:

$$\Lambda_{40} \approx 10^{10}$$

Chebyshev-Interpolation

Polynominterpolation: Chebyshev

Chebyshev-Polynome erster Art:

$$T_n(x) = \cos(n \arccos(x)), x \in [-1,1]$$

Chebyshev-Polynome zweiter Art:

$$U_n(x) = \frac{\sin((n+1) \arccos(x))}{\sin(\arccos(x))}, x \in [-1,1]$$

Sind orthogonal bezügl diskretes und stetiges Skalarprodukt (Diskret auf Chebyshev-Knoten):

$$(f, g) = \sum_{l=0}^n f(x_l)g(x_l)$$

$$\langle f, g \rangle = \int_{-1}^1 f(x)g(x) \frac{1}{\sqrt{1-x^2}} dx$$

Polynominterpolation: Chebyshev

Chebyshev-Knoten (Nullstellen von $T_{n+1}(x)$):

$$x_k = a + \frac{1}{2}(b - a) \left(\cos \left(\frac{2k+1}{2(n+1)} \pi \right) + 1 \right) \quad k = 0, \dots, n$$

Chebyshev-Abszissen (Extrema von $T_n(x)$):

$$x_k = a + \frac{1}{2}(b - a) \left(\cos \left(\frac{k}{n} \pi \right) + 1 \right) \quad k = 0, \dots, n$$

Beide sind sehr viel bessere Verteilungen als äquidistante Stützstellen.

Polynominterpolation: Chebyshev

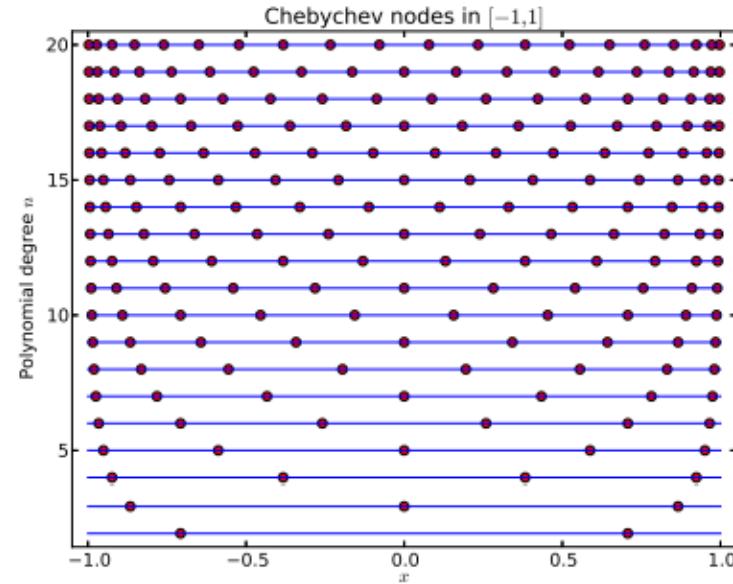
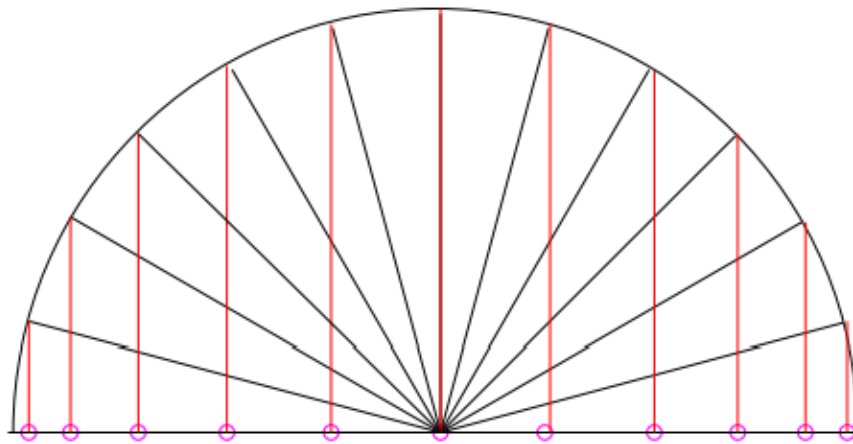


Figure 2.6: Chebyshev-Knoten: links für $n = 11$, rechts für verschiedene n

Polynominterpolation: Chebyshev

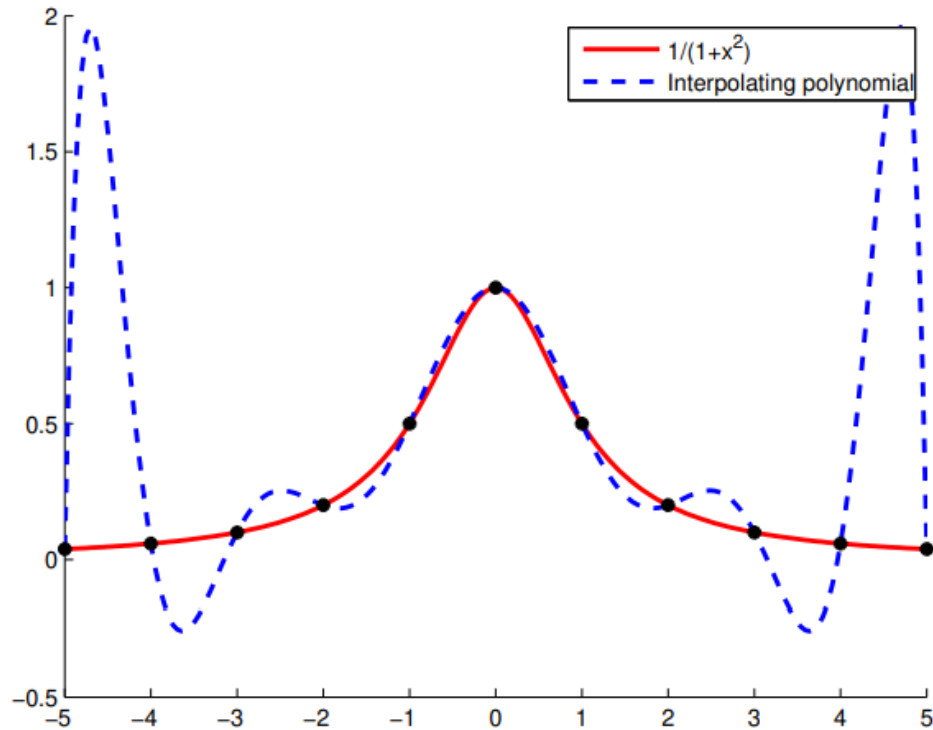


Figure 2.7: Äquidistante Stützstellen

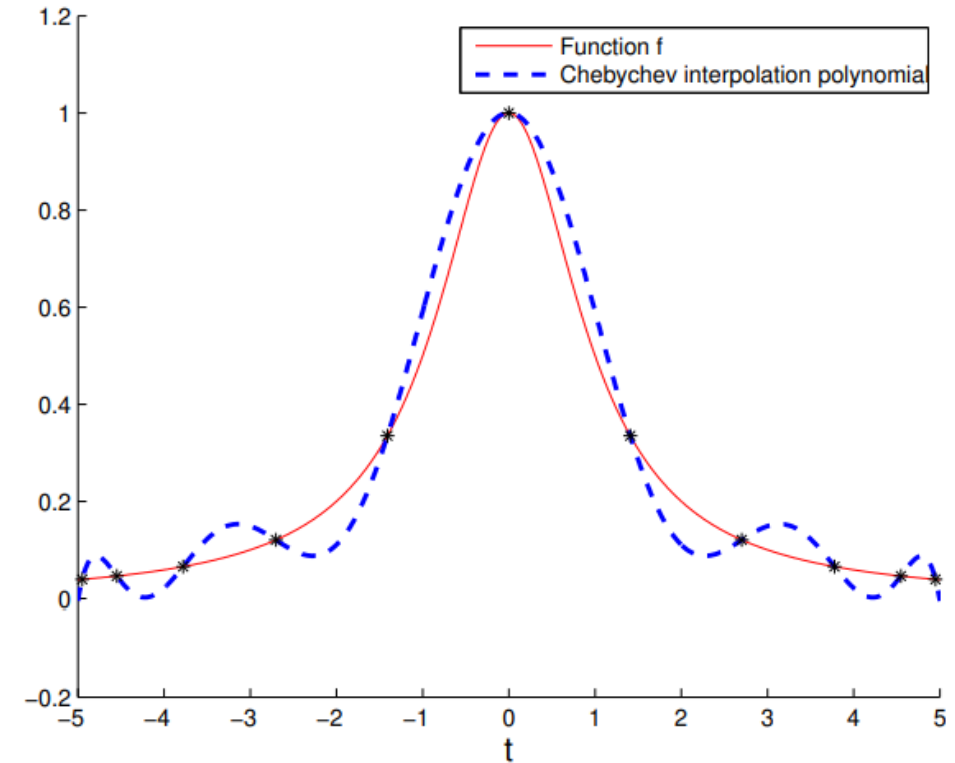


Figure 2.8: Chebyshev-Knoten

Lebesgue-Konstante bei Chebyshev

Wir erinnern uns für Äquidistant:

$$\Lambda_{40} \approx 10^{10}$$

Nun für Chebyshev:

$$\Lambda_n \approx \frac{4}{\pi^2} \log n \approx_{n=40} 145$$

Chebyshev-Knoten sind die **beste** Wahl der Stützstellen - **unabhängig** von der Interpolationsmethode!

Chebyshev-Interpolationspolynom

Natürlich gibt es auch eine Chebyshev-Interpolationsformel:

$$p(x) = c_0 + c_1 T_1(x) + \dots + c_n T_n(x)$$

Die Koeffizienten berechnen sich wie folgt:

$$c_0 = \frac{1}{n+1} \sum_{\ell=0}^n f(x_\ell)$$

Für $k = 1, \dots, n$:

$$c_k = \frac{2}{n+1} \sum_{\ell=0}^n f(x_\ell) T_k(x_\ell)$$

Chebyshev-Auswertung mit Clenshaw

Ziel: *Auswertung von $p(x) = c_0 + c_1T_1(x) + \dots + c_nT_n(x)$*

Rekursion (rückwärts):

$$d_{n+2} = d_{n+1} = 0$$

$$d_k = c_k + (2x)d_{k+1} - d_{k+2}$$

Auswertung:

$$p(x) = d_0 - xd_1 = \frac{1}{2}(d_0 - d_2)$$

Sehr stabil!!

Fazit

